

**Beyond BERT: Building Better Adaptive Generative AI Chatbot with Semantic Context  
Matching—Challenges, Solutions, and Insights**

Yaakov Sternberg and Andrew Roehr

University of San Diego

Natural Language Processing and GenAI (AAI-520-04)

Andrew Van Benschoten

October 21, 2024

# Challenges Faced and Solutions Implemented

## 1. Model Performance: Context Matching and Response Accuracy

Several challenges were encountered during the development of the Advanced Generative Chatbot, primarily around context matching, handling ambiguous questions, and ensuring response accuracy. Initially, relying on a standard BERT model without improved context matching led to inaccurate answers for broad questions. This issue was addressed by incorporating SBERT (all-MiniLM-L6-v2) for semantic similarity-based context retrieval, ensuring that the most relevant context was selected.

Handling confidence in answers was another challenge, as the BERT model often generated responses with low certainty. A confidence threshold of 0.4 was introduced to reject low-confidence predictions and provide a fallback message when appropriate. Additionally, ensuring the end index of the answer exceeded the start index resolved issues with improperly predicted answer spans.

## 2. Computational Resource Constraints and Training Runtimes

Writing code to successfully train a model was challenging, but the extremely long runtimes & resources required for training advanced models, particularly on the 1.7 GB Ubuntu dataset (attempted during the first few weeks of the project) posed significant challenges that made it even harder for several reasons:

- **Long training times prevented rapid iteration on model improvements.** Each time a change was made to the model or training parameters, it could take several hours or even days to see the results, making it difficult to assess the impact of adjustments in a timely

manner. This hindered the ability to refine models based on feedback and significantly delayed overall project progress.

- **Extended runtimes disrupted the workflow**, making it difficult to maintain focus, momentum, and a continuous train of thought. Long waits between model runs often led to breaks in concentration, forcing a re-orientation to the task at hand each time training results were available, which further slowed down the process.
- **Loss of training progress due to session terminations**: Session terminations were highly disruptive, as they resulted in complete loss of training progress. When a session was terminated due to issues like code bugs, insufficient memory, or internet disconnections (in Colab), the training would have to start over from the beginning. This significantly slowed the pace of the project, as valuable time was lost to re-running training processes that had already been underway.
- **Computational overhead and resource limits**: Managing limited computing resources was a significant hurdle, as well over 500 Colab compute units were used during the project, pushing the limits of available resources. This resource constraint made it challenging to optimize training configurations and explore different models without concern for running out of available compute time.

### *Solutions Implemented:*

- **Colab Pro hardware resources**, including A100 & L4 GPUs with 50+ GB RAM were critical to reducing run time for model training and contextual embeddings.
  - **Limits**: Training on the ~1.7 GB Ubuntu was still a long (and expensive) process.
- **Code optimization**: More efficient code to fully utilize hardware resources included:
  - **Increased batch size**: Allowed the GPU to process more samples at once.

- **Limits:** Poor generalization if it converges to sharp minima in loss landscape
- **Mixed Precision Training (FP16):** Cuts memory usage for increased efficiency.
  - **Limits:** Can reduce model accuracy due to lower numerical precision.
- **Optimized/Dynamic Learning Rate Scheduling:** Models converge faster by starting with a high learning rate (speed) and gradually decreasing it (accuracy).
  - **Limits:** Can miss optimal solutions or get stuck in suboptimal minima
- **Early Stopping:** I didn't automate this, but did write code to log progress and manually stopped as progress plateaued &/ began overfitting.
  - **Limits:** Models can suddenly progress in generalization after this point, à la “grokking” (Power et al., 2022; Zhang et al., 2017).
- **Running multiple concurrent sessions:** To speed up the process and maintain workflow, I tried running numerous versions & variations of the code simultaneously.
  - **Limits:** This became difficult to manage, keep track of, and keep all the variations in mind at the same time (kind of like playing multiple games of chess at once).  
 Additionally, Colab has limits to the number of simultaneous sessions that can be run at once. Finally, it did not solve, and in fact exacerbated, the issues of resource overhead/cost as well as progress being lost due to session terminations.
- **Testing Code On Smaller Subsets with less epochs:** Running & debugging on small subsets of the data with less epochs to ensure functionality before scaling up to the full dataset enabled faster testing and reduced the risk of long runtimes getting wasted due to unexpected issues.
  - **Limits:** Some issues only manifested when the code was run on the complete dataset, leading to additional troubleshooting during full-scale runs.

- **Saving checkpoints & results to Drive:** To address progress loss, code was written not only to save final results (such as tokenizers, embeddings, models, etc.) but also to frequently save checkpoints to Google Drive. Then to utilize this, additional code had to be written to check for these checkpoints and automatically resume from them, or use the saved results, when rerunning after a session terminated.
  - **Limits:** Implementing this solution presented challenges, particularly in ensuring checkpoint consistency. Running multiple versions of the code could result in overwrites or the use of results from previous, failed attempts. For instance, if a version of the code initially produced poor results, and then I tweaked it, I had to make sure that the new run did not inadvertently use those suboptimal outputs. This required careful management of checkpoints to ensure that only the desired results were loaded and used during each training session, as well as logging to easily see whether a run was using a checkpoint or training from scratch.
- **Smaller dataset:** Ultimately, due to time & resource constraints, we gradually realized that we had to switch to a smaller dataset (the Stanford Question Answering Dataset (SQuAD)), which required less computational resources and allowed for faster iterations. This change enabled more rapid testing of model adjustments and made it easier to focus on fine-tuning without the constant risk of running out of compute resources.
  - **Downside:** Much less data to train on, and not in dialog format.
- **Pretrained models:** The above challenges made it (even more) difficult to train a model from scratch, and in the few attempts that we did manage to complete training for didn't produce great results, so we ended up utilizing pre-trained models (while still also running code in the hopes get something decent trained in time.

### 3. User Interaction and Interface Development

**Challenge:** Facilitating seamless and multi-turn conversations required versatile user interaction modes, posing integration and performance challenges.

**Solutions Implemented:**

- **Developing Multiple Interfaces:** Created both a Flask web interface and a Jupyter Notebook interface using ipywidgets to cater to different user preferences and interaction scenarios. Additionally we used Gardio to create a web-based option.
- **Optimizing User Experience:** Ensured that the interfaces supported smooth multi-turn conversations by implementing efficient backend processes and responsive frontend designs.

### Model Architecture and Rationale Behind Chosen Design

The architecture utilizes two key models: BERT for Question Answering and SBERT for context matching.

The BERT variant was fine-tuned on SQuAD to identify answer spans within a given context. Its dual start and end logits enable the extraction of precise answers, making it suitable for span-based QA tasks (Devlin J., Et al. 2019).

BERT was chosen for efficient semantic similarity matching to select the most relevant context. Compared to simple TF-IDF matching, SBERT offers better semantic understanding for user queries, improving the relevance of the selected context.

The system follows a two-step process: First, SBERT identifies the most appropriate context based on the user's query. Then, BERT extracts the answer span from the selected context. This

modular approach ensures scalability and the ability to swap out components independently, such as upgrading to more advanced BERT variants or SBERT models.

## Evaluation Results and User Feedback

Initial evaluation of our BERT-based Question Answering (QA) model, enhanced with SBERT for context retrieval, was conducted using a subset of 1000 samples from the SQuAD v2.0 development set. Various implementations of the code resulted in a range of Exact Match (EM) scores from 27-40%, indicating that it can accurately identify the exact answer in approximately 27-40% of cases. Meanwhile, the F1 score, which considers the overlap between predicted and ground-truth answers by calculating the harmonic mean of precision and recall, was in the low 40s%.

These results demonstrated some effectiveness of transfer learning in natural language processing tasks, leveraging pre-trained BERT models for both question answering and context matching to benefit from knowledge acquired on large-scale datasets. These metrics suggest a reasonable balance between precision and recall in answer extraction, however, the higher F1 score compared to the EM score suggests that while the model effectively captures partial correctness, it still struggles to consistently provide exact matches.

[Code was written to compare various models](#), all of which were in the same range.

Comparison of Models:

|   | Model                                             | EM    | F1        | Precision | Recall    | No Answer Accuracy | Latency (ms) | Confidence (%) | BLEU (%)  | ROUGE-1 F1 (%) | ROUGE-2 F1 (%) | ROUGE-L F1 (%) |
|---|---------------------------------------------------|-------|-----------|-----------|-----------|--------------------|--------------|----------------|-----------|----------------|----------------|----------------|
| 0 | distilbert-base-uncased-distilled-squad           | 35.60 | 40.757736 | 38.566719 | 40.643274 | 4.112337           | 6.475815     | 78.409790      | 39.557497 | 41.015039      | 23.457684      | 41.015039      |
| 1 | bert-large-uncased-whole-word-masking-finetune... | 38.85 | 43.181303 | 41.794367 | 43.382242 | 2.407222           | 15.842991    | 77.813016      | 42.204091 | 43.436610      | 24.720110      | 43.436610      |
| 2 | deepset/roberta-base-squad2                       | 39.70 | 42.063155 | 41.735947 | 43.019603 | 0.702106           | 10.776973    | 82.243594      | 41.361977 | 42.177194      | 22.890491      | 42.177194      |
| 3 | deepset/xlm-roberta-large-squad2                  | 38.85 | 41.401271 | 40.934684 | 42.130417 | 0.702106           | 19.954375    | 89.108491      | 40.734798 | 41.511592      | 22.539195      | 41.505710      |

However, we then [switched](#) to using the official `evaluate-v2.0.py` script and [Hugging Face 'evaluate' library](#) got much higher results due to it assigning an empty string ("") for unanswerable questions instead of relying on the model to generate specific responses:

"exact": 87.83795165501559,

"f1": 92.55862751229617,

In terms of actual use, the model often got things wrong, as illustrated in the following example:

**SQuAD v2.0 Chatbot using BERT and SBERT**

Chatbot

When was George Washington born?

1753

When did George Washington die?

99

How old was George Washington when he died?

21

Washington in fact lived from 1732 to 1799.

## Future Improvements and Scalability Options

**Model Upgrade:** Replacing the existing BERT model with more advanced alternatives such as DistilBERT, GPT-3.5, or a BERT-base fine-tuned with SQuAD v2.0 could significantly enhance answer accuracy, reduce inference time, and improve overall system efficiency. DistilBERT offers a lighter architecture for faster inference, GPT-3.5 provides superior language understanding, and fine-tuning BERT with domain-specific data like SQuAD v2.0 can boost accuracy in specialized areas. Selecting the right model will depend on the specific needs of the deployment environment, balancing speed and understanding.

**Caching Mechanism for Context Embeddings:** Another key optimization would involve implementing a caching mechanism for context embeddings. This approach would store the SBERT-generated embeddings of frequently used contexts, minimizing redundant calculations and significantly improving response times. This enhancement would be especially useful for production environments where multiple users may ask similar questions repeatedly.

**Multi-Turn Conversation Support:** Finally, introducing multi-turn conversation support through dialogue state tracking would allow the chatbot to retain context over several interactions, making conversations more coherent and natural. This capability would enable the chatbot to handle follow-up questions effectively, refer back to previous answers, and manage ambiguous queries more gracefully. Together, these improvements would enhance both the performance and user experience, making the chatbot more robust, scalable, and responsive in real-world applications. While it may require additional resources for managing state and memory, strategies like state compression or cloud-based storage could mitigate these challenges, ensuring scalability and a seamless user experience.

**Enhanced Data Utilization:** Fine-tuning on larger datasets would likely improve results.

## References

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint*, arXiv:1810.04805.  
<https://arxiv.org/abs/1810.04805>
- Jurafsky, D., & Martin, J. H. (2024). *Speech and Language Processing* (3rd ed., draft).
- Power, A., Burda, Y., Edwards, H., Babuschkin, I., & Misra, V. (2022). *Grokking: Generalization beyond overfitting on small algorithmic datasets*. *arXiv preprint*, arXiv:2201.02177.  
<https://arxiv.org/abs/2201.02177>
- Tunstall, L., Von Werra, L., & Wolf, T. (2022). *Natural language processing with transformers*. O'Reilly Media.
- Zhang, C., Bengio, S., Hardt, M., Recht, B., & Vinyals, O. (2017). Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations* [Conference paper], *ICLR 2017*. <https://arxiv.org/abs/1611.03530>

